MEOP Process
Renovation Status and Toolbox Guide

Current software snapshot, operating model, batch workflow, and roadmap

Purpose. This document summarizes what the renovated MEOP toolbox currently does, how it is installed and used, how runtime data are expected to be found, how to run the new resumable multi-deployment workflow, what remains to be validated scientifically across deployments, and the recommended strategy for the next development phases.

Snapshot date	April 17, 2026
Current objective	Reach a minimal, functional Python version of the MEOP processing chain that can be re-run over the available deployments with readable logs, resumable state, and updated summary tables
Main regression examples	`ct88-225-12`, `ct78-465-12`, and the shortened `ct96-24-13` full-resolution reference set
Pipeline now covered in Python	Catalog discovery, raw ODV import, `lr0`, QC/filtering, `hr0`, `hr1/lr1`, `fr0`, `fr1`, `hr2`, diagnostics, and resumable batch processing
Execution model	Single Python execution path; no alternate runtime backend remains in the package
Tests in repository	65 collected tests in the current snapshot

Prepared for the MEOP toolbox renovation effort

Contents

1 Executive summary

The toolbox has moved from an operator workspace to a Python package with explicit runtime ownership and a nearly complete processing path for the currently targeted products. The package can now bootstrap its runtime layout, discover deployments and tags from CSV and JSON metadata, import raw ODV files, generate the low-resolution and high-resolution profile products, generate full-resolution products from HR text files, produce standard diagnostics, and run through many deployments in sequence while logging failures without stopping the whole run.

The most important practical change in this phase is operational rather than architectural: there is now a resumable batch runner suitable for reprocessing all available deployments. It keeps a persistent per-deployment state file, skips deployments that already completed successfully only while their canonical outputs still exist unless forced, records one log file per deployment, generates readable Markdown and CSV run reports, reconciles stale successful state entries against the filesystem at startup, and refreshes `list_tags.csv` and `list_deployments.csv` incrementally at the end of the run.

At this point the project is close to a minimal functional version. The next work is less about scaffolding and more about scientific validation, richer diagnostics, and the remaining publication/reporting helpers.

2 General description

2.1 Scientific intent

MEOP Process transforms marine mammal CTD observations into Argo-like products and associated diagnostics that can be inspected, adjusted, summarized, and released. The software has to combine heterogeneous metadata sources, low-resolution ODV inputs, optional high-resolution time series, and operator-managed catalog and JSON metadata.

2.2 Software intent of the renovated toolbox

The renovated toolbox now has four simultaneous roles:

1. a Python package and CLI that can be installed, imported, and scripted in a standard way;
2. a single Python processing engine for the current workflow;
3. a runtime data manager that owns where tables, catalogs, raw data, outputs, and batch state live;

3 Current status of the toolbox

3.1 Pipeline ownership

Workflow element	Owner	Notes
Configuration loading and path resolution	Python	No external runtime dependency.
Runtime data layout and validation	Python	Explicit directories under the canonical `data/` tree plus the configured `public/` release tree.

Workflow element	Owner	Notes
Catalog and mirrored JSON metadata resolution	Python	Uses `list_deployment.csv`, `list_deployment_hr.csv`, and mirrored deployment/platform JSON files.
Raw ODV import and profile indexing	Python	Deployment and tag discovery can be driven directly from raw ODV content.
`create_ncargo` / `lr0`	Python	Includes low-resolution netCDF assembly.
`lr0` QC and filtering	Python	Ported to Python.
Location adjustment	Python	Applies crawl/CLS/SMRU-based geographic corrections and updates profile metadata.
`hr0`	Python	Regular 1 dbar vertical product.
`apply_adjustments`	Python	Uses runtime coefficient, parameter, and salinity-offset tables.
`hr1/lr1` no-TLC branch	Python	Implemented and regression-tested.
`hr1/lr1` TLC branch	Python	Uses `gsw` when available and is regression-tested on reference tags.
`fr0`	Python	Detects ascents from HR text files, associates low-resolution metadata, and writes full-resolution products.
`fr1`	Python	Applies the same adjustment/TLC logic in the FR path.
`hr2`	Python	Uses `fr1` when FR data exist and falls back to `hr1` otherwise.
Standard diagnostics figures	Python	Overview plus section plots are now available.
Batch processing and summary-table refresh	Python	Resumable multi-deployment runner with logs and reports.
High-end analyst diagnostics and publication redesign	Future work	Still to be designed in depth.

3.2 Reference cases currently used

The current regression set is built around three representative cases:

- `ct88-225-12`: low-resolution and TLC validation without full-resolution data;
- `ct78-465-12`: a second low-resolution reference to increase confidence in the non-FR path;
- `ct96-24-13`: shortened full-resolution reference set used to validate `fr0`, `fr1`, `traj`, and `hr2`.

These cases are valuable because they test both the low-resolution/high-resolution workflow and the branch that uses full-resolution HR text inputs.

4 What has been completed

4.1 Repository and package structure

The project now uses a standard `src/` layout with `pyproject.toml`, a package root at `src/meop_process/`, a public API module, a CLI, and compatibility wrappers under `python/`. The historical entry point `python/meop_process.py` still exists, but it now forwards to the modern package rather than acting as the implementation itself.

4.2 Runtime data ownership

The package now owns its runtime layout explicitly. In practice this means that the code no longer relies on implicit knowledge of where files are stored. It has named roots for:

- packaged CSV defaults;
- operator-maintained catalogs;
- mirrored JSON metadata;
- raw ODV inputs;
- raw HR CTD inputs;
- processed profile and trajectory outputs;
- plots, diagnostics, and maps;
- batch logs, reports, and resumable state.

4.3 Scientific workflow stages already ported

The following steps are already owned by Python and exercised in tests:

- deployment and tag resolution;
- raw ODV import and parsing;
- `lr0`;
- the low-resolution QC/filter block;
- `hr0`;
- `apply_adjustments`;
- `hr1` and `lr1` for both no-TLC and TLC branches;
- `fr0`;
- `fr1`;
- `hr2`;
- standard diagnostics figures.

4.4 Operational batch runner

A dedicated multi-deployment runner is now present. Its design goals are:

- continue past deployment-level failures;
- make reruns cheap by skipping deployments that already completed successfully;
- produce one log file per deployment;
- produce a run-level Markdown summary and CSV table;
- reconcile stale successful state entries when outputs have been deleted;
- refresh summary metadata tables at the end without reopening every netCDF file when nothing changed.

5 Installation

5.1 Editable install

The recommended installation remains a standard editable install:

```
python -m pip install -e .
```

5.2 Expected Python dependencies

The current minimal scientific runtime expects at least:

- `numpy`
- `pandas`
- `xarray`
- `scipy`
- `h5netcdf`
- `h5py`
- `matplotlib`
- `gsw`

If map backgrounds are desired in diagnostics, `cartopy` can be installed as an optional extra. The code currently falls back to a plain longitude/latitude plot when `cartopy` is unavailable.

6 Runtime data layout

6.1 Canonical runtime roots

- **Packaged/default tables:** `src/meop_process/resources/tables/`, mirrored into `data/tables/`.
- **Operator-maintained catalogs:** `data/catalog/`, notably `list_deployment.csv` and `list_deployment_hr.csv`.
- **Mirrored metadata JSON:** `data/data_raw/config_files/`, for example `deployment2.json`, `platform2.json`, and patch files.
- **Low-resolution raw inputs:** `data/data_raw/raw_smru_data_odv/`.
- **High-resolution raw inputs:** `data/data_raw/raw_smru_hr_data/<year>/`, containing files such as `<prefix><instr_id>_ctd.txt`.
- **Processed profile outputs:** `data/data_prof/`, containing `lr0`, `lr1`, `hr0`, `hr1`, `hr2`, `fr0`, and `fr1` profile products.
- **Processed trajectory outputs:** `data/data_traj/`.
- **Diagnostics by tag:** `data/plots_by_tags/`.
- **Deployment and overview plots:** `data/plots_by_deployments/` and `data/plots_overview/`.
- **Maps:** `data/maps/`.
- **Batch state and reports:** `data/batch/`.

6.2 How raw data are expected to be found

Low-resolution data are staged directly under `data/data_raw/raw_smru_data_odv/`. High-resolution files are resolved from `list_deployment_hr.csv` and are expected under the runtime HR tree, typically as files named like `<optional_prefix><instr_id>_ctd.txt` inside the appropriate year directory.

This is now explicit in Python and does not rely on any alternate runtime.

7 Usage

7.1 Single deployment processing

Installed entry point:

```
meop-process --deployment ct88 --process_data --diagnostics
```

Repository wrapper:

```
python python/meop_process.py --deployment ct88 --process_data --diagnostics
```

7.2 Batch processing over all deployments

Installed entry point:

```
meop-process --run-all-deployments
```

Dedicated batch entry point:

```
meop-process-batch
```

Repository wrapper:

```
python scripts/run_all_deployments.py
```

Useful variants:

```
meop-process --run-all-deployments --diagnostics
meop-process --run-all-deployments --force-failed
meop-process --run-all-deployments --force
meop-process --run-all-deployments --deployment ct96
meop-process --run-all-deployments --jobs 8 --verbose
python scripts/run_all_deployments.py --diagnostics
python scripts/run_all_deployments.py --force-failed
python scripts/run_all_deployments.py --force
python scripts/run_all_deployments.py --deployment ct96
python scripts/run_all_deployments.py --jobs 8 --verbose
```

7.3 What the batch runner writes

By default the runner writes under `data/batch/`:

- `latest/deployment_status.json`: persistent deployment-level state;
- `runs/<timestamp>/logs/<deployment>.log`: one log per deployment;
- `runs/<timestamp>/summary.md`: human-readable run report;

- `runs/<timestamp>/summary.csv`: machine-readable run table.

A deployment marked successful is skipped on the next batch run only while its canonical outputs still exist, unless a force option is used. At batch startup, stale successful entries whose outputs have disappeared are pruned from `latest/deployment_status.json` before skip decisions are made.

7.4 Metadata summary tables

At the end of a batch run the package refreshes `list_tags.csv` and `list_deployments.csv`. The refresh is incremental:

- deployments processed in the current run are refreshed;
- deployments whose tag inventory changed are refreshed;
- unchanged deployments are left untouched.

This makes reruns much cheaper once the main outputs already exist.

8 Test cases and validation strategy

8.1 Current regression philosophy

The current test suite is built around a small number of carefully chosen reference cases rather than a full global comparison across every historical product. The goal has been to make the Python workflow operational first, then broaden scientific validation deployment by deployment.

8.2 Current test coverage

The repository currently contains 65 collected tests covering:

- configuration loading and data-layout bootstrap;
- raw ODV import and catalog resolution;
- `ct88` `lr0/lr1/hr0/hr1` regressions;
- `ct78` low-resolution and no-FR `hr2` regression;
- shortened `ct96` `fr0/fr1/hr2/traj` regression;
- diagnostics output generation;
- metadata summary refresh;
- resumable batch-run behavior.

8.3 What still needs validation

The next validation phase should use the real local environment to compare the Python outputs against existing reference products over a wider set of deployments. The important point is that the current package is now structured so that such comparisons can be made systematically rather than manually.

9 API overview

The stable public surface is intentionally small and centered in `src/meop_process/api.py`. The most relevant functions at the current stage are:

- `bootstrap_data_store`
- `describe_runtime_data_layout`
- `validate_runtime_data_layout`
- `load_info_deployment`
- `import_raw_data`
- `process_tags`
- `apply_adjustments`
- `create_fr0`
- `create_hr2`
- `generate_diagnostics`
- `update_metadata_summaries`
- `run_all_deployments`

The design intent is to keep this surface stable while allowing the implementation below it to continue evolving.

10 What remains to be done

10.1 Scientific and operational work still open

The main remaining items are now concentrated in a few areas:

- broader scientific validation over many deployments;
- richer diagnostic and analyst-facing comparison tools against historical profiles and climatologies;
- redesign of the publication/reporting helpers;
- broader scientific robustness work for remaining problematic deployments.

10.2 Why diagnostics remain important

Although a standard Python diagnostics set now exists, the future diagnostic layer is still a major work package. It should eventually help an analyst inspect profiles, compare them with reference datasets, understand the consequences of proposed adjustments, and archive the reasoning used during delayed-mode processing.

11 Planned strategy for future developments

The recommended next steps are:

1. use the new batch runner to process as many deployments as practical in the real environment;
2. review the per-deployment logs and batch summary to identify systematic failures;
3. compare Python outputs with reference outputs on a representative set of deployments;
4. strengthen the metadata summaries and diagnostics where operational review shows they are still too light;

5. redesign the reporting and publication layer in Python once the scientific core is trusted.

The important change is that the project is now in a position where large-scale testing is practical. Earlier phases were about creating the package and porting key workflow stages. The current phase is about exercising that workflow widely, collecting evidence, and using that evidence to prioritize the next scientific and operational refinements.

12 Conclusion

The toolbox is now close to a minimal functional version. It can be installed as a Python package, run over deployments through a single execution path, resume large processing campaigns, produce readable logs and reports, and refresh the main summary tables efficiently. The next work should focus on scientific confidence and analyst workflow quality rather than on basic software scaffolding.